Автономная некоммерческая организация высшего образования

«Московский университет «СИНЕРГИЯ»

Факультет информационных систем и технологий

Дисциплина: «Основы методологии DevOps»

РЕФЕРАТИВНАЯ РАБОТА

на тему:

«Основы методологии DevOps и применение технологий CI/CD в современной разработке программного обеспечения»

Выполнил: Лоор Идальго Жан Пьер

Группа: ДБО-301рпо

Проверил: _____

Москва, 2026

Содержание

Введение 3
1. Понятие и сущность методологии DevOps 4
2. Понятие CI/CD и его роль в современной разработке 5
3. Основные этапы CI/CD-пайплайна 7
4. Инструменты CI/CD 10
    4.1. Системы управления исходным кодом и планированием 10
    4.2. Инструменты сборки 11
    4.3. Инструменты автоматического тестирования 11
    4.4. Контейнеризация, оркестрация и инфраструктура как код 12
    4.5. Мониторинг и эксплуатация 13
5. Преимущества и проблемы внедрения CI/CD 14
6. Пример практического CI/CD-пайплайна 15
Заключение 17
Список использованных источников 18

Введение

Современная разработка программного обеспечения требует высокой скорости выпуска новых версий, стабильности работы приложений и быстрого исправления ошибок. В условиях постоянного изменения требований пользователей и бизнеса традиционные подходы к разработке не всегда позволяют эффективно управлять процессами создания, тестирования и развертывания программных продуктов.

Одним из ключевых направлений развития современной инженерии программного обеспечения является методология DevOps. Она направлена на объединение разработки, тестирования и эксплуатации в единый непрерывный процесс. DevOps помогает сократить время между написанием кода и его доставкой конечному пользователю, а также повысить надежность и предсказуемость программных систем.

Особое место в DevOps занимает технология CI/CD. CI/CD включает в себя непрерывную интеграцию, непрерывную доставку и непрерывное развертывание программного обеспечения. Данные подходы позволяют автоматизировать сборку, тестирование и выпуск приложений, снизить количество ошибок, ускорить процесс разработки и повысить качество программного продукта.

Актуальность данной темы связана с тем, что CI/CD используется в современной промышленной разработке веб-приложений, мобильных приложений, микросервисов и облачных систем. Разработчики, тестировщики, DevOps-инженеры и специалисты по эксплуатации должны понимать, как устроены CI/CD-пайплайны, какие инструменты применяются на разных этапах и какие преимущества дает автоматизация процессов разработки.

Цель работы - рассмотреть основы методологии DevOps и изучить роль технологий CI/CD в современной разработке программного обеспечения.

Для достижения поставленной цели необходимо решить следующие задачи: рассмотреть понятие и сущность методологии DevOps; изучить понятие CI/CD и его основные компоненты; описать этапы CI/CD-пайплайна; рассмотреть основные инструменты, применяемые в CI/CD; проанализировать преимущества и возможные проблемы внедрения CI/CD; привести пример практического CI/CD-пайплайна для современного приложения.

Объектом исследования является процесс разработки и развертывания программного обеспечения. Предметом исследования являются технологии CI/CD как часть методологии DevOps.

1. Понятие и сущность методологии DevOps

DevOps представляет собой современный подход к организации разработки программного обеспечения, который объединяет процессы разработки, тестирования, развертывания и эксплуатации. Сам термин DevOps образован от английских слов Development и Operations, то есть «разработка» и «эксплуатация».

Главная идея DevOps заключается в устранении разрыва между командами разработчиков и специалистами, отвечающими за эксплуатацию программных систем. В традиционной модели разработки программисты создают приложение, после чего передают его администраторам или инженерам эксплуатации. Такой подход часто приводит к задержкам, ошибкам при развертывании и проблемам взаимодействия между командами.

Методология DevOps предполагает тесное сотрудничество всех участников процесса разработки. Разработчики, тестировщики, системные администраторы, DevOps-инженеры и специалисты по безопасности работают в единой среде и используют общие инструменты автоматизации. Это позволяет быстрее обнаруживать ошибки, оперативно выпускать обновления и поддерживать стабильную работу программного продукта.

К основным принципам DevOps относятся автоматизация, непрерывная интеграция, непрерывная доставка, мониторинг, обратная связь и культура совместной ответственности. Автоматизация позволяет минимизировать ручные действия и снизить вероятность человеческих ошибок. Непрерывная интеграция помогает регулярно объединять изменения кода. Непрерывная доставка обеспечивает готовность приложения к выпуску. Мониторинг позволяет отслеживать состояние системы после развертывания.

DevOps также связан с изменением культуры работы команды. Важным становится не только написание кода, но и понимание всего жизненного цикла программного продукта: от планирования задачи до эксплуатации приложения пользователями. Благодаря этому команда несет ответственность не только за разработку, но и за качество, стабильность и безопасность конечного продукта.

Таким образом, DevOps можно рассматривать как совокупность методологических, организационных и технических практик, направленных на ускорение разработки, повышение качества программного обеспечения и улучшение взаимодействия между участниками IT-процесса.

2. Понятие CI/CD и его роль в современной разработке

CI/CD является одной из ключевых практик методологии DevOps. Данная технология направлена на автоматизацию процессов интеграции, проверки, доставки и развертывания программного обеспечения. Благодаря CI/CD команда разработки может быстрее выпускать новые версии продукта, уменьшать количество ошибок и поддерживать стабильность приложения.

Аббревиатура CI/CD включает несколько понятий. CI означает Continuous Integration, то есть непрерывную интеграцию. CD может означать Continuous Delivery - непрерывную доставку, а также Continuous Deployment - непрерывное развертывание.

Непрерывная интеграция представляет собой практику, при которой разработчики регулярно добавляют изменения кода в общий репозиторий. После каждого изменения автоматически запускаются процессы сборки и тестирования. Это позволяет быстро обнаружить ошибки, несовместимость кода или нарушение логики работы программы. Чем раньше ошибка обнаружена, тем дешевле и быстрее ее исправить.

Непрерывная доставка означает, что после успешной сборки и прохождения тестов программный продукт автоматически подготавливается к выпуску. При этом окончательное решение о развертывании новой версии может принимать человек. Такой подход позволяет компании всегда иметь готовую к выпуску версию приложения.

Непрерывное развертывание является более автоматизированным вариантом. В этом случае все изменения, которые успешно прошли сборку, тестирование и проверки качества, автоматически попадают в рабочую среду. Это позволяет выпускать обновления очень быстро, но требует высокого уровня автоматизации, надежных тестов и хорошо настроенного мониторинга.

Роль CI/CD в современной разработке заключается в том, что данная технология делает процесс выпуска программного обеспечения более предсказуемым и управляемым. Вместо ручной сборки, ручного тестирования и ручного развертывания команда использует автоматизированный pipeline. Такой pipeline выполняет одинаковые действия при каждом изменении кода, что снижает влияние человеческого фактора.

CI/CD особенно важно для команд, которые работают над сложными веб-приложениями, мобильными приложениями, микросервисами и облачными системами. В таких проектах изменения могут вноситься ежедневно или даже несколько раз в день. Без автоматизации процесс проверки и выпуска новых версий становится медленным и рискованным.

Таким образом, CI/CD можно рассматривать как техническую основу DevOps-подхода. Если DevOps определяет общую культуру взаимодействия и ответственности команды, то CI/CD предоставляет конкретные инструменты и процессы для автоматизации жизненного цикла программного обеспечения.

3. Основные этапы CI/CD-пайплайна

CI/CD-пайплайн представляет собой последовательность автоматизированных этапов, через которые проходит программный код от момента написания до развертывания и дальнейшего мониторинга. Такой pipeline позволяет стандартизировать процесс разработки и сделать его более надежным.

В приложенных преподавателем материалах CI/CD представлен как непрерывный цикл, включающий этапы Plan, Code, Build, Test, Release, Deploy, Operate и Monitor. Вокруг данного цикла указаны различные инструменты: GitHub, GitLab, Jira, Confluence, Gradle, Bazel, Webpack, Jest, JUnit, Jenkins, Docker, Kubernetes, Terraform, Prometheus, Datadog, Argo CD и другие [1].

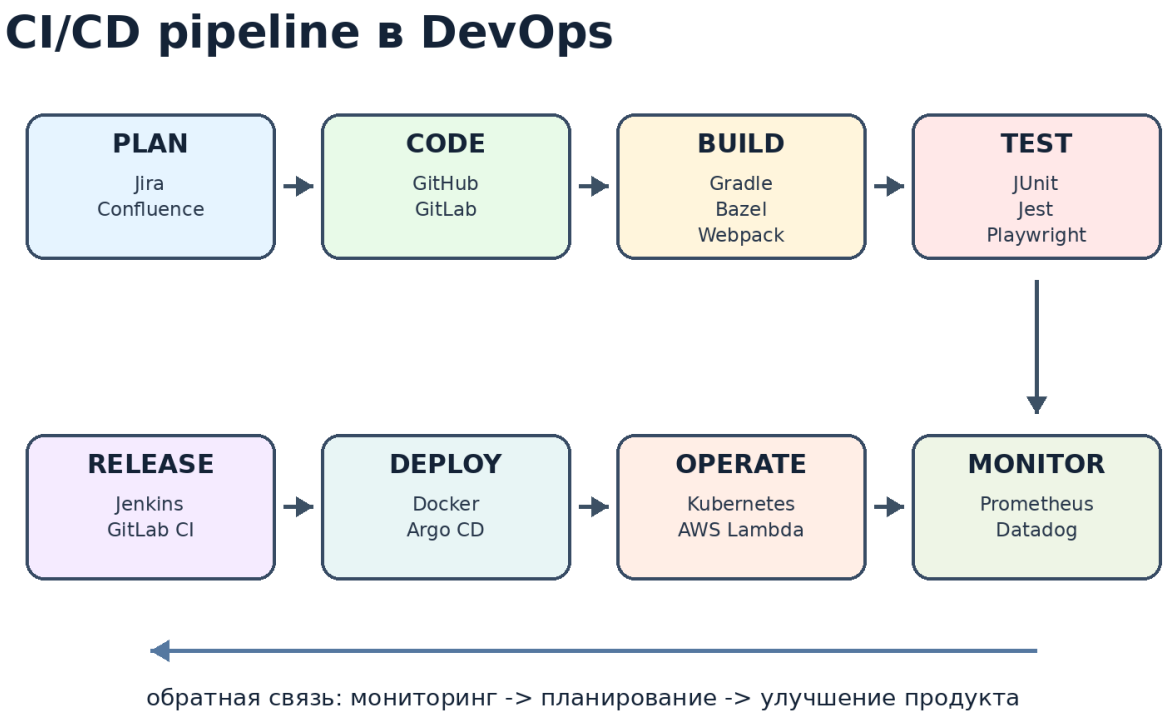

Рисунок 1 - Обобщенная схема CI/CD-пайплайна в DevOps. Источник: составлено автором на основе материалов задания.

Таблица 1 - Основные этапы CI/CD-пайплайна

Этап	Содержание этапа	Примеры инструментов
Plan	Планирование задач, требований и сроков разработки.	Jira, Confluence
Code	Написание кода, работа с ветками, хранение исходных файлов.	GitHub, GitLab
Build	Сборка приложения и подготовка артефактов.	Gradle, Bazel, Webpack
Test	Автоматическая проверка качества и корректности кода.	JUnit, Jest, Playwright
Release	Подготовка версии приложения к выпуску.	Jenkins, GitLab CI
Deploy	Развертывание приложения в тестовой или рабочей среде.	Docker, Kubernetes, Argo CD
Operate	Эксплуатация приложения, управление ресурсами и стабильностью.	Kubernetes, AWS Lambda
Monitor	Наблюдение за метриками, ошибками и доступностью сервиса.	Prometheus, Datadog

Этап планирования связан с определением задач, требований и сроков разработки. На данном этапе команда формулирует, какие изменения необходимо внести в программный продукт. Для планирования могут

использоваться системы управления задачами и документацией, например Jira и Confluence.

Этап написания кода предполагает непосредственную работу разработчиков над программным продуктом. Код обычно хранится в системе контроля версий, например GitHub или GitLab. Разработчики создают отдельные ветки, вносят изменения, выполняют проверку и затем отправляют код в общий репозиторий.

Этап сборки необходим для подготовки приложения к запуску. На этом этапе исходный код преобразуется в исполняемую или готовую к использованию форму. Для сборки могут применяться Gradle, Bazel или Webpack. Например, Gradle часто используется для автоматизации сборки Java-проектов, а Webpack применяется во frontend-разработке [2; 5].

Этап тестирования является одним из самых важных этапов CI/CD. После сборки автоматически запускаются тесты, которые проверяют корректность работы программы. Это могут быть unit-тесты, интеграционные тесты, end-to-end тесты и другие виды проверок. Для тестирования могут использоваться JUnit, Jest, Playwright и другие инструменты [6; 7; 8].

Этап развертывания связан с переносом приложения в среду выполнения. Развертывание может выполняться в тестовой, staging- или production-среде. В современных проектах часто применяется контейнеризация, так как она позволяет запускать приложение в одинаковом окружении независимо от сервера.

Этап мониторинга необходим для наблюдения за состоянием приложения и инфраструктуры. С помощью мониторинга команда получает информацию о производительности, ошибках, нагрузке, доступности сервисов и других показателях. Для мониторинга могут использоваться Prometheus, Datadog и другие системы [12; 13; 14].

4. Инструменты CI/CD

Инструменты CI/CD можно разделить на несколько групп. Каждая группа отвечает за определенную часть жизненного цикла программного продукта. В приложенных материалах преподавателя представлены технологии, которые относятся к разным этапам pipeline: системы управления кодом, средства сборки, инструменты тестирования, системы автоматизации, контейнеризации, оркестрации, инфраструктуры как кода и мониторинга [1].

Таблица 2 - Основные группы инструментов CI/CD

Группа инструментов	Примеры	Роль в CI/CD
Управление кодом	GitHub, GitLab	Хранение исходного кода, pull request, merge request, запуск pipeline.
Планирование	Jira, Confluence	Управление задачами, требованиями и документацией проекта.
Сборка	Gradle, Bazel, Webpack	Автоматизация сборки, подготовка артефактов приложения.
Тестирование	JUnit, Jest, Playwright	Автоматическая проверка качества и работоспособности кода.
CI/CD-серверы	Jenkins, GitLab CI, Buildkite	Организация автоматических pipeline сборки, тестирования и релиза.
Контейнеризация	Docker, BuildKit	Создание контейнерных образов приложения.
Оркестрация	Kubernetes	Управление контейнерами и масштабирование приложений.
GitOps	Argo CD	Автоматическое развертывание на основе состояния Git-репозитория.
Инфраструктура как код	Terraform	Описание и управление инфраструктурой с помощью кода.
Мониторинг	Prometheus, Datadog	Сбор метрик, контроль доступности и анализ состояния системы.

4.1. Системы управления исходным кодом и планированием

Системы управления исходным кодом являются основой CI/CD. GitHub и GitLab позволяют хранить код проекта, организовывать работу с ветками, проводить code review и фиксировать историю изменений. При внесении изменений в репозиторий может автоматически запускаться pipeline, который

собирает проект, запускает тесты и подготавливает приложение к развертыванию.

Инструменты планирования, такие как Jira и Confluence, не выполняют сборку кода напрямую, но играют важную организационную роль. Jira используется для управления задачами, спринтами и статусами работ. Confluence позволяет хранить техническую документацию, требования, инструкции и архитектурные решения. Благодаря этому CI/CD становится частью управляемого процесса разработки, а не набором отдельных скриптов.

4.2. Инструменты сборки

Сборка является обязательным этапом CI/CD, так как исходный код должен быть преобразован в готовую форму. Для Java-проектов часто применяется Gradle. Он позволяет описывать зависимости, задачи сборки, запуск тестов и формирование артефактов. В крупных проектах важным становится сокращение времени сборки, так как длительная сборка замедляет обратную связь для разработчиков [2; 3].

Bazel применяется для быстрой и воспроизводимой сборки сложных проектов. Его преимущество заключается в том, что он может эффективно кэшировать результаты и пересобирать только необходимые части проекта. Такой подход особенно полезен для больших кодовых баз и многомодульных систем [4].

Webpack используется преимущественно во frontend-разработке. Он собирает JavaScript, TypeScript, CSS и другие ресурсы в оптимизированные файлы, которые можно использовать в веб-приложении. В CI/CD Webpack часто запускается автоматически после изменения frontend-кода [5].

4.3. Инструменты автоматического тестирования

Тестирование в CI/CD обеспечивает проверку качества программного продукта до попадания изменений в рабочую среду. Unit-тесты позволяют

проверять отдельные функции и модули. Интеграционные тесты проверяют взаимодействие между компонентами. End-to-end тесты имитируют действия пользователя и проверяют приложение целиком.

JUnit является одним из распространенных фреймворков для тестирования Java-приложений. Он позволяет создавать автоматические тесты, которые запускаются при каждом изменении кода [7; 8]. Для JavaScript и TypeScript часто используется Jest. Он подходит для unit-тестирования frontend- и backend-кода, а также интегрируется с системами CI [6].

Playwright применяется для end-to-end тестирования веб-приложений. С его помощью можно автоматически открывать браузер, выполнять пользовательские действия и проверять результат. Использование таких тестов в CI/CD помогает выявлять ошибки интерфейса до релиза [9].

4.4. Контейнеризация, оркестрация и инфраструктура как код

Контейнеризация является важной частью современных CI/CD-процессов. Docker позволяет упаковать приложение вместе с зависимостями в контейнерный образ. Такой образ можно запускать в разных средах одинаковым образом, что уменьшает риск ошибки из-за различий между компьютером разработчика, тестовым сервером и production-средой.

BuildKit используется для более быстрой и гибкой сборки контейнерных образов. В CI/CD это важно, потому что сборка образов может занимать значительное время. Оптимизация сборки позволяет быстрее получать обратную связь и ускорять релизный процесс [11].

Kubernetes используется для управления контейнерами в распределенной инфраструктуре. Он позволяет запускать приложения, масштабировать их, перезапускать при сбоях и управлять конфигурацией. В CI/CD Kubernetes часто выступает целевой средой развертывания [16].

Terraform относится к подходу Infrastructure as Code. С его помощью инфраструктура описывается в виде файлов конфигурации. Это позволяет создавать и изменять серверы, сети, базы данных и другие ресурсы автоматически, а также хранить описание инфраструктуры в системе контроля версий [15].

Argo CD применяется для GitOps-подхода. В GitOps желаемое состояние приложения хранится в Git-репозитории, а специальная система автоматически синхронизирует реальную инфраструктуру с этим состоянием. Такой подход повышает прозрачность изменений и упрощает откат к предыдущей версии [18].

4.5. Мониторинг и эксплуатация

После развертывания приложения CI/CD-процесс не заканчивается. Команда должна понимать, как приложение работает в реальной среде. Для этого используются системы мониторинга, сбора логов и анализа метрик.

Prometheus предназначен для сбора метрик и наблюдения за состоянием сервисов. Он позволяет отслеживать нагрузку, время ответа, количество ошибок, доступность компонентов и другие показатели. На основе метрик можно строить графики и настраивать оповещения [13; 14].

Datadog является комплексной платформой мониторинга, которая помогает анализировать состояние приложений, инфраструктуры и логов. Использование мониторинга позволяет быстрее выявлять проблемы после релиза и принимать решения о доработке продукта [12].

Важным элементом эксплуатации является обратная связь. Если после релиза мониторинг показывает рост ошибок или ухудшение производительности, команда может быстро откатить изменения или выпустить исправление. Таким образом, мониторинг связывает этап эксплуатации с новым этапом планирования.

5. Преимущества и проблемы внедрения CI/CD

Внедрение CI/CD дает организации значительные преимущества, однако требует технической подготовки, дисциплины команды и изменения процессов разработки. Для объективной оценки необходимо рассмотреть не только положительные эффекты, но и возможные трудности.

Таблица 3 - Преимущества и проблемы внедрения CI/CD

Аспект	Преимущество	Потенциальная проблема
Скорость разработки	Изменения быстрее проходят путь от кода до релиза.	Требуется время на настройку pipeline.
Качество кода	Автоматические тесты выявляют ошибки на раннем этапе.	Нужно поддерживать тесты в актуальном состоянии.
Надежность релиза	Процесс выпуска становится повторяемым и предсказуемым.	Ошибочная автоматизация может быстро распространить дефект.
Командная работа	Разработчики, тестировщики и эксплуатация работают в едином процессе.	Необходима культура ответственности и взаимодействия.
Инфраструктура	Инфраструктура описывается кодом и воспроизводится автоматически.	Сложность настройки IaC и управления секретами.
Мониторинг	Команда быстрее видит проблемы после релиза.	Нужно правильно выбирать метрики и пороги оповещений.

Главное преимущество CI/CD заключается в ускорении обратной связи. Разработчик быстро узнает, что его изменение не прошло тесты или нарушило сборку. Это позволяет исправлять ошибку до того, как она попадет в релиз.

Вторым важным преимуществом является повторяемость процесса. Ручное развертывание зависит от человека, его опыта и внимательности. Автоматизированный pipeline выполняет одни и те же действия одинаковым способом, что снижает вероятность случайных ошибок.

Еще одним преимуществом является повышение прозрачности. В CI/CD видно, какое изменение было внесено, кто его сделал, какие тесты были запущены, какой образ был собран и в какую среду он был развернут. Это упрощает аудит и поиск причин неисправностей.

Однако внедрение CI/CD может быть сложным. Команде необходимо настроить репозитории, pipeline, тесты, сборку, хранение артефактов, секреты, инфраструктуру и мониторинг. Если автоматизация настроена

неправильно, она может создать дополнительные риски. Например, недостаточное покрытие тестами может привести к тому, что дефект автоматически попадет в production.

Следовательно, CI/CD требует не только инструментов, но и зрелого инженерного подхода: качественных тестов, понятной архитектуры, контроля доступа, безопасного хранения секретов и готовности команды поддерживать pipeline как часть продукта.

6. Пример практического CI/CD-пайплайна

Для лучшего понимания технологии рассмотрим пример CI/CD-пайплайна для веб-приложения. Такой пример может быть применен к приложению, написанному на JavaScript, TypeScript, Java или другом языке программирования. Конкретный стек может отличаться, но общая логика остается одинаковой.

1. Разработчик создает новую ветку в GitHub или GitLab и выполняет задачу из системы планирования.

2. После внесения изменений код отправляется в удаленный репозиторий.

3. CI-система автоматически запускает pipeline.

4. Выполняется установка зависимостей и сборка проекта.

5. Запускаются unit-тесты с использованием Jest или JUnit.

6. При успешном прохождении тестов создается Docker-образ.

7. Образ публикуется в container registry.

8. Приложение разворачивается в Kubernetes или другой среде.

9. Argo CD синхронизирует состояние приложения с Git-репозиторием.

10. Prometheus и Datadog собирают метрики и помогают отслеживать состояние приложения.

Ниже представлен упрощенный пример pipeline в формате YAML. Он не является универсальным для всех проектов, но показывает типовую последовательность действий: установка зависимостей, тестирование, сборка и подготовка к развертыванию.

```
stages:
  - install
  - test
  - build
  - deploy

install_dependencies:
  stage: install
  script:
    - npm install

automated_tests:
  stage: test
  script:
    - npm test

build_application:
  stage: build
  script:
    - npm run build
    - docker build -t web-app:latest .

deploy_to_environment:
  stage: deploy
  script:
    - echo "Deploy application to Kubernetes or staging server"
  when: manual
```

В данном примере pipeline разделен на четыре стадии. На первой стадии устанавливаются зависимости проекта. На второй стадии запускаются автоматические тесты. На третьей стадии выполняется сборка приложения и контейнерного образа. На четвертой стадии выполняется развертывание. В реальном проекте также могут добавляться анализ качества кода, проверка безопасности, публикация артефактов, миграции базы данных и уведомления команды.

Практическая ценность такого pipeline заключается в том, что он делает процесс выпуска приложения контролируемым. Разработчику не нужно вручную запускать все команды и проверять каждый шаг. Система автоматически сообщает, на каком этапе возникла ошибка, а команда может быстро принять решение о исправлении, откате или выпуске новой версии.

Заключение

В ходе выполнения реферативной работы были рассмотрены основы методологии DevOps и применение технологий CI/CD в современной разработке программного обеспечения. Было установлено, что DevOps представляет собой не только набор технических инструментов, но и подход к организации командной работы, основанный на сотрудничестве, автоматизации и совместной ответственности за результат.

CI/CD является важной частью DevOps, так как обеспечивает автоматизацию ключевых этапов жизненного цикла программного продукта: интеграции кода, сборки, тестирования, подготовки релиза, развертывания и мониторинга. Непрерывная интеграция помогает быстро выявлять ошибки, непрерывная доставка обеспечивает готовность продукта к выпуску, а непрерывное развертывание позволяет автоматически доставлять изменения пользователям.

В работе были рассмотрены основные этапы CI/CD-пайплайна: Plan, Code, Build, Test, Release, Deploy, Operate и Monitor. Также были проанализированы инструменты, которые применяются на этих этапах: GitHub, GitLab, Jira, Confluence, Gradle, Bazel, Webpack, JUnit, Jest, Playwright, Jenkins, Docker, BuildKit, Kubernetes, Terraform, AWS Lambda, Argo CD, Prometheus и Datadog.

Особое значение имеет то, что CI/CD повышает скорость и надежность разработки, делает релизы более предсказуемыми, снижает количество ручных операций и помогает команде быстрее получать обратную связь. При этом внедрение CI/CD требует качественной настройки pipeline, автоматических тестов, мониторинга, контроля доступа и инженерной дисциплины.

Таким образом, цель работы достигнута: были изучены основы DevOps и технологии CI/CD, рассмотрены этапы CI/CD-пайплайна, основные

инструменты и практический пример применения данной технологии в современной разработке программного обеспечения.

Список использованных источников

1. Материалы преподавателя по теме «CICD» с перечнем рекомендуемых ресурсов и схемой CI/CD-пайплайна.

2. Gradle - система автоматизации сборки Java-проектов // Хабр. URL: https://habr.com/ru/hubs/gradle/posts/ (дата обращения: 27.04.2026).

3. 37 000 unit-тестов против Gradle: как мы добились 12-минутного прогона // Хабр. URL: https://habr.com/ru/companies/alfa/articles/993352/ (дата обращения: 27.04.2026).

4. Сборка с Bazel в реальном проекте // Хабр. URL: https://habr.com/ru/companies/ntechlab/articles/540728/ (дата обращения: 27.04.2026).

5. Webpack: руководство для начинающих // Хабр. URL: https://habr.com/ru/articles/514838/ (дата обращения: 27.04.2026).

6. Jest. Официальная документация. URL: https://jestjs.io/ru/ (дата обращения: 27.04.2026).

7. JUnit: что это такое за фреймворк Java // Skillfactory. URL: https://blog.skillfactory.ru/glossary/junit/ (дата обращения: 27.04.2026).

8. Туториал по JUnit 5 - Введение // Хабр. URL: https://habr.com/ru/articles/590607/ (дата обращения: 27.04.2026).

9. Playwright: лучшая альтернатива Selenium. Первое знакомство // Хабр. URL: https://habr.com/ru/companies/amvera/articles/834846/ (дата обращения: 27.04.2026).

10. Основы CI/CD. Знакомство с Jenkins // Хабр. URL: https://habr.com/ru/companies/slurm/articles/691876/ (дата обращения: 27.04.2026).

11. Jenkins. Официальный сайт. URL: https://www.jenkins.io/ (дата обращения: 27.04.2026).

12. BuildKit в Kubernetes: мануал по быстрой и автомасштабируемой сборке проектов // Хабр. URL: https://habr.com/ru/companies/kts/articles/960922/ (дата обращения: 27.04.2026).

13. Использование систем мониторинга Datadog при разработке проекта на Azure // Хабр. URL: https://habr.com/ru/companies/Social_Discovery_Group/articles/598599/ (дата обращения: 27.04.2026).

14. Человеческим языком про метрики 2: Prometheus // Хабр. URL: https://habr.com/ru/companies/tochka/articles/685636/ (дата обращения: 27.04.2026).

15. Полное руководство по Prometheus // Хабр. URL: https://habr.com/ru/companies/slurm/articles/455290/ (дата обращения: 27.04.2026).

16. Основы Terraform // Хабр. URL: https://habr.com/ru/companies/otus/articles/696694/ (дата обращения: 27.04.2026).

17. Kubernetes на пальцах: самое простое объяснение, что это такое // Хабр. URL: https://habr.com/ru/companies/selectel/articles/956266/ (дата обращения: 27.04.2026).

18. AWS Lambda - теория, знакомство // Хабр. URL: https://habr.com/ru/articles/457100/ (дата обращения: 27.04.2026).

19. GitOps-платформа на базе Argo CD // Хабр. URL: https://habr.com/ru/companies/kts/articles/723760/ (дата обращения: 27.04.2026).